



本科生实验报告

实验课程：编译原理实验

实验名称：自动生成语法分析程序

专业名称：计算机科学与技术

实验地点：东校园实验中心大楼 B202

报告时间：2026/6/4

学号	23320143
姓名	唐雅妍
邮箱	tangyy77@mail2.sysu.edu.cn

目录

1	实验目的	3
2	配置和试用 JavaCUP	3
3	修改词法程序	3
4	符号表与类型设计	4
4.1	环境与作用域管理	4
4.2	符号种类体系	5
5	优先级处理	7
6	异常处理	7
6.1	错误类型总览	7
6.2	各类异常的检测逻辑	8
6.2.1	语法类异常 (SyntacticException)	8
6.2.2	语义异常 (SemanticException)	9
6.3	错误恢复策略	10
7	调用图生成	11
7.1	总体思路	11
7.2	过程节点注册	11
7.3	调用点处理	12
7.4	建边	12
8	讨论不同生成工具的差异	13
8.1	JavaCUP 和 GNU Bison	13
8.2	JavaCUP 和 JavaCC	14
9	自动测试	14

10 附录

14

1 实验目的

下载一个语法分析程序自动生成工具 JavaCUP,利用该工具自动产生一个 Oberon-0 语言的语法分析和语法制导翻译程序,进行异常处理和调用图生成。

2 配置和试用 JavaCUP

由于原来那个网站挂了,找了一个CUP User's Manual作为替代。该网站介绍了 CUP 工具的使用,并提供一个简易计算器的实现,但是由于该网站比较陈旧,使用了 `new Integer(i_val)` 和 `DefaultSymbolFactory()` 等弃用的函数,需要修改为 `Integer.valueOf` 和 `ComplexSymbolFactory`。

CUP 可以通过 `-parser` 和 `-symbols` 来指定生成的类名称。值得注意的是,由于代码编译和运行都依赖 `java_cup.runtime` 包,因此编译和运行都需要指定 `runtime` 的 jar 包,生成代码使用主包。并且 `symbol` 不能命名为 `Symbol` 类,会与 jar 包冲突。

在进行语法分析,需要词法分析联动,可以通过 `new Parser(scanner())` 实现,并在 CUP 中重载构造函数即可。

观察提供的 cup 代码可以发现,他提供了 `precedence left PLUS` 来对优先级进行控制,并可以使用 `%prec UMINUS` 来控制当前产生式的优先级,每条产生式的优先级为其最后一个终结符。

结果如图 1 所示,可以看到安装正确。

3 修改词法程序

利用上一个实验的词法分析程序,对于 `jflex`,他提供了 `%cup`,会自动适应 `init` 和 `scan` 的接口。

词法分析需要生成 `java_cup.runtime.Symbol` 的类型,cup 能够自动给每个非终结符和终结符计算左边和右边位置,对于命名为 `e1` 的终结符,可以通过 `e1left` 和 `e1right` 得到其位置,为 `int` 类型。因此只能提供行号或者字符偏移量,为了方便阅读,这里选择行号进行初始化。

```

ex3 > simple_test > test.txt
1  1+2;
2  3+4*(5+6);
3  2+5;

问题 3  输出  调试控制台  终端  端口  评论

===== Test Complete =====
Press any key to continue . . .
(base) PS E:\github-res\tyylab3\ex3\simple_test> run.bat

E:\github-res\tyylab3\ex3\simple_test>REM filepath: e:\github-res\tyylab3\ex3\run.bat
Active code page: 65001
===== Running Parser =====

Please type your arithmetic expression:
3
47
7

===== Test Complete =====
Press any key to continue . . .

```

图 1: CUP 简易例子

cup 会自动根据 cup 文件和 flex 文件定义的终结符生成 symbol 文件，如果指定了该文件的名称，需要修改 flex 为 `sym.xxx` 以适配接口。

对于 Identifier 和 Integer 需要额外传具体的值。

4 符号表与类型设计

符号表和类型系统类图如图 2 所示。

4.1 环境与作用域管理

符号表采用链式作用域结构，由 `mySymbol.Env` 类实现。每个 `Env` 持有当前作用域名称、本地符号映射表，以及指向父作用域的引用，形成一条从内层到外层的查找链。

表 1: `mySymbol.Env` 核心方法

方法 / 属性	说明
<code>enterEnv(name)</code>	压栈：创建子作用域，设定其父亲，并进入新 <code>Env</code>
<code>exitEnv()</code>	出栈：恢复当前环境为父 <code>Env</code>
<code>addSymbol_s(id, sym)</code>	向当前环境插入符号，为静态类
<code>lookupLocal(id)</code>	仅在当前层查找符号（用于过程调用解析）
<code>lookup(id)</code>	沿作用域链向上逐层查找，支持嵌套过程访问外层变量
<code>toString()</code>	返回从根到当前层的全路径名，如 <code>ModA.ProcB</code> ，用于调用图节点命名

在解析器构造函数中，全局环境的符号表预置了 `INTEGER`、`BOOLEAN` 和 `ERROR` 三种

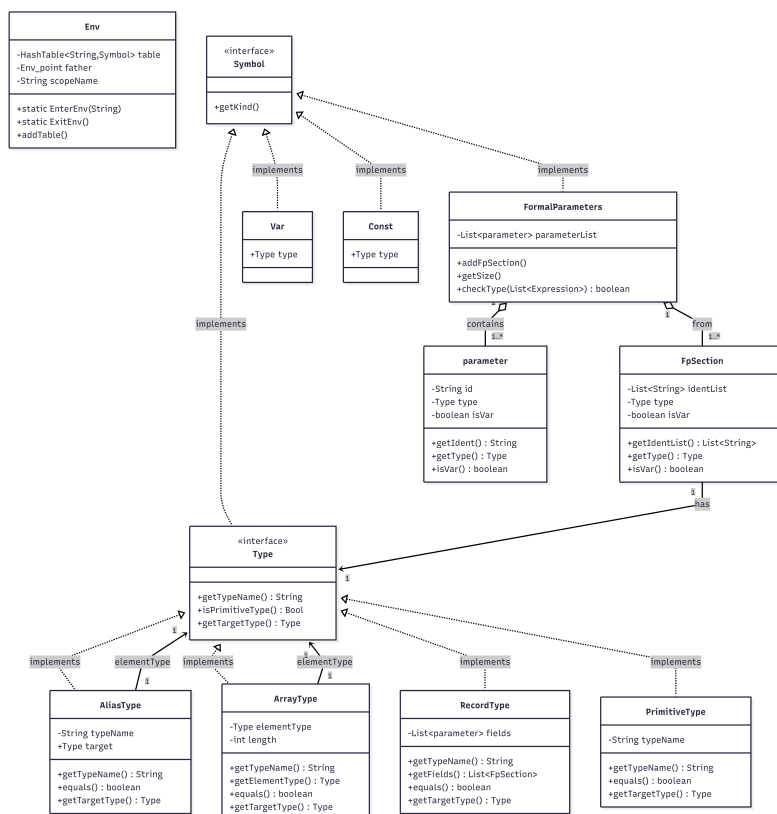


图 2: 符号表及类型系统类图

基本类型（为 `PrimitiveType`），以及 `READ`、`WRITE`、`WRITELN` 三个内置过程的形参描述，使它们在所有作用域可见。他们与用户自定义的函数和类型一起处理，简化流程。

```
public static final PrimitiveType INTEGER = new
    PrimitiveType("INTEGER");
```

Listing 1: 预定义标识符

4.2 符号种类体系

所有符号继承自公共基类 `TableSymbol`，并通过 `getKind()` 方法区分种类。

- **Var**: 变量声明，携带类型信息；可作为赋值左值
- **Const**: 常量声明；仅可读，不可作为赋值左值
- **函数参数定义**: `FpSection` 是由于 Oberon-0 的定义为同一类型一起定义（如 `Gcd(x,y: INTEGER)`），但参数列表匹配时需要每个参数逐一匹配，因此设定两个数据结构，`parameter` 用于最后存储。
- **类型匹配**: 由于有别名，需要每个类型均有 `getTargetType()` 穿透到底层具体类型，并且重载 `equals()` 函数，比较前先调用 `getTargetType()`，确保别名透明。

- **AliasType**: 与 C 语言的 typedef 一致, 对别名进行重命名时必须已经申明过该别名, 因此在 AliasType 的构造函数中, 会展开别名的别名, 如代码 2 所示。
- **ArrayType**: 存储元素类型, 类型匹配时判断元素类型是否相等。
- **RecordType**: 记录类型, 按字段名管理多个 Parameter, 类型匹配时只有在是同一个类型声明时才相等。

```
public AliasType(String typeName, Type targetType) {
    this.typeName = typeName;
    if(targetType instanceof AliasType) {
        this.targetType = ((AliasType) targetType).getTargetType();
    } else {
        this.targetType = targetType;
    }
}
@Override
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (!(obj instanceof AliasType)) return false;
    AliasType other = (AliasType) obj;
    return targetType.equals(other.targetType);
}
```

Listing 2: AliasType

选择器链求值 对于记录类型和数组类型, 支持任意深度嵌套, 如 `a[i].field[j]`。对选择器节点序列逐步解引用: 字段选择器 `(.)` 要求当前类型为 `RecordType` 并查找字段; 下标选择器 `([])` 要求当前类型为 `ArrayType` 并取出元素类型。每步均先做 `AliasType` 穿透。

函数参数列表校验 `FormalParameters` 类负责保存过程的参数列表, 并提供 `checkType` 方法执行完整校验:

- **数量检查**: 实参数量与形参数量不一致时, 上游触发 `ParameterMismatchedException`。
- **类型检查**: 逐一比较每个实参的 `getTargetType()` 与对应形参类型, 不一致时返回类型描述字符串, 上游触发 `TypeMismatchedException`。
- **VAR 参数**: 标记为 `VAR` 的形参要求实参必须是左值 (`isLValue() == true`), 否则报语义错误。

```
===== Running Parser =====  
-(1)  
(-1) + (3)  
-(1 * 2)  
-(-1 * 2)  
===== Test Complete =====
```

```
===== Running Parser =====  
(NOT D) AND (E)  
(NOT D AND E) = (F)  
TypeMismatchException at line 6: Type mismatch in '=': expected INTEGER
```

图 3: 部分优先级显示

5 优先级处理

根据 Oberon-0 EBNF 文法的定义, 得到优先级如 3 所示。因此可以直接展开文法写为 $E ::= E \text{ PLUS } E$, 从而简化代码实现。

```
precedence right ASSIGN;  
precedence left EQ, NEQ, LT, LEQ, GT, GEQ;  
precedence left PLUS, MINUS, OR;  
precedence right UMINUS, UPLUS;  
precedence left TIMES, DIV, MOD, AND;  
precedence left LPAREN;  
precedence right NOT;  
  
PLUS expression %prec UPLUS //定义一元优先级
```

Listing 3: 优先级定义

实验测试在 $-1 + 3$, $--1 * 2$ 以及 $d \& e = f$ 的语句下优先级是否选取正确, 结果如图 3 所示, 可以看到能够正确处理优先级, 符合 Oberon-0 文法。

6 异常处理

6.1 错误类型总览

实验中通过 `ErrorReport.reportError(ErrorType, line, message)` 统一上报各类错误, `ErrorType` 枚举类型如表 2 所示。

表 2: 错误类型枚举及触发场景

错误类型	典型触发场景
Syntactic	模块/过程名首尾不匹配；缺少分号；赋值语句缺少右侧表达式
Semantic	未定义标识符；标识符非类型/变量/常量；赋值左值非变量；调用未定义过程
TypeMismatched	运算符两侧类型不符； IF/WHILE 条件非 BOOLEAN ；赋值两侧类型不一致；数组索引不为 INT ；实参类型错误
ParameterMismatched	用户定义过程的实参数量与形参数量不符
MissingOperator	两个整数字面量相邻；预定义函数参数数量错误
MissingOperand	算符缺少左/右操作数； NOT 缺少操作数；空括号对
MissingLeftParenthesis	形参列表或过程调用中，) 前无匹配 (
MissingRightParenthesis	过程调用中 (后遇到非法记号，期望)

6.2 各类异常的检测逻辑

6.2.1 语法类异常 (SyntacticException)

CUP 语法规则本身完成大部分结构检测，以下情形需要动作代码配合：

名称一致性校验 在 `module` 和 `procedure_body` 的结束规则中，比对开始标识符与 **END** 后的标识符是否相同。不一致时上报错误，解析继续。

缺少分号 `procedure_part` 包含”过程声明后无 **SEMI**”的备选产生式，匹配后立即上报并正常处理下一个过程：

```
procedure_part ::=
procedure_part procedure_declaration:pp {
    ErrorReport.reportError(SyntacticException, ppright,
        "Missing semicolon after procedure declaration");
    :}
```

赋值缺少右值 `assignment` 规则含以下备选产生式，匹配后报告缺失表达式：

```
assignment ::=
identifier_selector:is ASSIGN {
    ErrorReport.reportError(SyntacticException, isright,
        "Missing expression in assignment"); :}
```

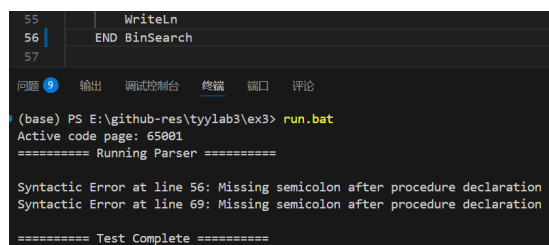
左括号和右括号匹配 在缺少左括号时, 不能直接通过匹配 `expression RPAREN` 来识别, 当 `func(expression, expression` 下一个符号遇到括号时, LALR(1) 会产生归约移入冲突。而 CUP 提供 `error RPAREN` 来对无法匹配任何产生式时跳过任意个字符到右括号进行恢复。

在缺少右括号时, 会比较麻烦。对于函数调用 (如 `Gcd(x, y);`), 可以匹配错误模式 `LPAREN:l expression_list_opt:e1`, 函数调用的后继符号一定为分号, 因此不会产生冲突。

对于普通表达式, 在 `((expression))` 的情况下, LALR(1) 只能向前看一个符号, 会产生冲突。对于函数申明时, 形式参数列表使用分号分隔, `fun(a, b: INTEGER; c: INTEGER);` 参数列表内部存在分号, 而形式参数声明结束后紧接着也会出现分号。因此当分析器读到分号时, 仅凭一个向前看符号无法判断。

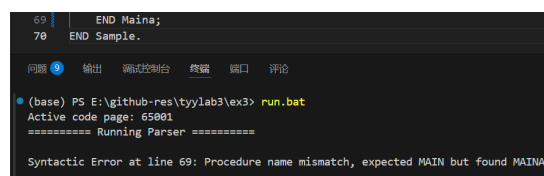
缺少操作数 在 `expression` 规则中, 通过大量形如 `OP expression` 和 `expression OP` 的产生式捕获缺失操作数情形, 并返回 `errorType` 哨兵表达式, 避免级联错误。

```
/* 缺少左操作数示例 */
| EQ expression:e1
/* 两整数相邻——缺少运算符 */
| INTEGER:i INTEGER
```



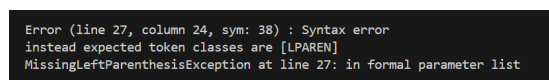
```
55      WriteLn
56      END BinSearch
57
(base) PS E:\github-res\tyylab3\ex3> run.bat
Active code page: 65001
===== Running Parser =====
Syntactic Error at line 56: Missing semicolon after procedure declaration
Syntactic Error at line 69: Missing semicolon after procedure declaration
===== Test Complete =====
```

(a) 缺少分号



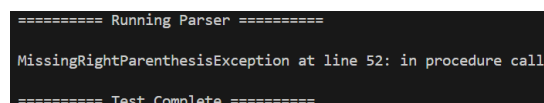
```
68      END Maina;
69      END Sample.
(base) PS E:\github-res\tyylab3\ex3> run.bat
Active code page: 65001
===== Running Parser =====
Syntactic Error at line 69: Procedure name mismatch, expected MAIN but found MAINA
```

(b) 名称一致性校验



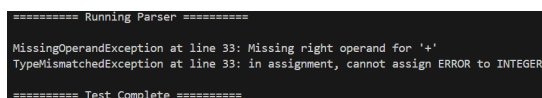
```
Error (line 27, column 24, sym: 38) : Syntax error
instead expected token classes are [LPAREN]
MissingLeftParenthesisException at line 27: in formal parameter list
```

(c) 缺少左括号



```
===== Running Parser =====
MissingRightParenthesisException at line 52: in procedure call
===== Test Complete =====
```

(d) 缺少右括号



```
===== Running Parser =====
MissingOperandException at line 33: Missing right operand for '+'
TypeMismatchedException at line 33: in assignment, cannot assign ERROR to INTEGER
===== Test Complete =====
```

(e) 缺少操作符

图 4: 语法分析部分结果

6.2.2 语义异常 (SemanticException)

实现了对标识符定义、类型, 左值等多种异常识别及错误恢复。

- **未定义标识符**: 在 `type` 规则和 `identifier selector` 规则中, 若 `lookup(id)` 返回 `null` 则报错, 并将结果类型设为 `ErrorType` (哨兵值), 使后续规则跳过进一步检测, 避免级联报错。
- **非法使用**: `identifier_selector` 中判断符号的 `kind`: 仅 `VAR` / `CONST` 合法, 其余 (如 `FORMALPARAMETERS`) 触发语义异常。
- **左值检查**: `assignment` 规则检查 `is.isLValue()`; `Const` 和表达式计算结果的 `isLValue` 均为 `false`, 对其赋值时报错。对于函数形参声明为 `VAR`, 也要判断实参是否为左值。
- **选择器类型错误**: 在处理选择器链时, 若对非 `RecordType` 使用 `.`, 或对非 `ArrayType` 使用 `[]`, 均触发 `SemanticException`。

所有运算、控制结构条件及赋值均在归约时即时做类型校验。

- **算术运算符** (`+` `-` `*` `DIV` `MOD`): 要求两操作数均为 `INTEGER`。
- **比较运算符** (`=` `#` `<` `<=` `>` `>=`): 要求两操作数均为 `INTEGER`, 结果类型为 `BOOLEAN`。
- **逻辑运算符** (`AND` `OR` `NOT`): 要求操作数为 `BOOLEAN`。
- **控制结构条件**: `IF`、`ELSIF`、`WHILE` 的条件表达式须为 `BOOLEAN` 类型, 判断在各自的内嵌动作中完成。

部分语义分析异常处理结果如图 5 所示。

```

===== Running Parser =====
TypeMismatchedException at line 67: For parameter "x", expected INTEGER, got BOOLEAN
===== Test Complete =====

```

(a) 类型匹配错误

```

===== Running Parser =====
TypeMismatchedException at line 33: Array index must be of type INTEGER
===== Test Complete =====

```

(b) 数组索引表达式错误

```

===== Running Parser =====
Error : Undefined field I
TypeMismatchedException at line 33: For parameter "value", expected INTEGER, got ERROR
Error : Index selector applied to non-array type
TypeMismatchedException at line 33: For parameter "value", expected INTEGER, got ERROR
===== Test Complete =====

```

(c) 记录 and 数组引用错误

图 5: 语义分析部分结果

6.3 错误恢复策略

解析器采用以下多层错误恢复手段, 力求在单次解析中发现尽可能多的错误:

- **哨兵类型**: 错误表达式统一返回 `errorType`。后续规则在做类型比较前可先判断操作数是否为 `errorType`, 若是则跳过, 防止同一错误的级联传播。

```

===== Running Parser =====
TypeMismatchedException at line 33: For parameter "value", expected INTEGER, got NODE = RECORD
ID: INTEGER;
VAL: INTEGER;
END
SemanticException at line 33: Index selector applied to non-array type
TypeMismatchedException at line 33: For parameter "value", expected INTEGER, got ERROR
SemanticException at line 41: Undefined field "V" in record type
TypeMismatchedException at line 41: Type mismatch in '>=': expected INTEGER
TypeMismatchedException at line 41: Condition expression in IF statement must be BOOLEAN
SyntacticException at line 56: Missing semicolon after procedure declaration
===== Test Complete =====

```

图 6: 错误恢复

- **CUP 内置错误记号**: 在 `formal_parameters`、`actual_parameters_opt` 等规则中使用 `error` 产生式，令解析器在遇到无法匹配的记号时跳过至同步点（如右括号），恢复后继续解析。
- **非致命错误不抛异常**: `ErrorReport.reportError` 仅记录并打印错误，不抛出异常，保证解析过程整体继续。
- **延迟解析**: 对当前作用域不可见的过程调用不立即报错，而是加入 `pendingEdges` 列表；所有过程声明完成后再统一解析，解决前向引用问题（详见 §7）。

如图 6 所示，该程序有多个语义语法错误，能够一一指出。

7 调用图生成

7.1 总体思路

调用图（`CallGraph`）由两类节点和有向边构成：

- **Procedure 节点**: 每个模块或过程，以全路径名（如 `ModA`、`ModA.Proc1`）标识。
- **CallSite 节点**: 每次过程调用，以唯一 ID（`cs_0`，`cs_1`，...）标识，存储可读的调用代码字符串（如 `foo(x, y+1);`）。
- **有向边**: 从 `CallSite` 节点指向被调过程节点，表示调用关系。

解析器维护 `callSiteCounter` 为每个调用点生成全局唯一 ID；同时也存储延迟边（`pendingEdges`），存储解析时尚未确定被调方的调用记录。

7.2 过程节点注册

每当进入新作用域时，`graph.addProcedure` 将其注册为调用图节点，进入过程时以全路径名注册节点，标签包含完整签名字符串，如 `"ModA.Proc1(VAR x: INTEGER)"`。全路径名能够使所有过程节点有唯一 ID，用于后续延迟边匹配调用点。

7.3 调用点处理

`procedure_call` 规则是调用图构建的核心，依次执行以下步骤：

表 3: 调用点处理流程

步骤	操作	说明
1	生成调用代码字符串	拼接 <code>foo(arg1, arg2, ...)</code> 形式的字符串，作为 Call-Site 节点标签
2	作用域链查找	从当前 Env 向上逐层 <code>lookupLocal</code> ，找到 <code>FORMALPARAMETERS</code> 即确定被调方
3a	立即建边（已找到）	参数类型校验通过后， <code>addCallSite + addEdge</code> ，完成图的即时更新
3b	延迟建边（未找到）	加入 <code>pendingEdges</code> ，记录调用 Env、参数列表、行号及被调名称
4	过滤内置过程	<code>READ / WRITE / WRITELN</code> 不加入调用图，仅做类型检查

7.4 建边

所有规则归约完成后，调用 `buildCallGraph(is_show)` 对 `pendingEdges` 统一处理：

1. 遍历每条 `PendingEdges`，从其记录的调用者 Env 重新沿作用域链向上查找被调过程名。
2. 找到后调用 `checkTypesWithMessage` 进行参数类型/数量校验，并调用 `graph.addEdge` 建立有向边。
3. 未找到时上报 `SemanticException`（未定义过程）。
4. 若 `is_show` 为 `true`，最终调用 `graph.show()` 输出调用图。

延迟建边机制解决了 Oberon 中过程可能在调用点之后才声明的前向引用问题，同时将参数校验逻辑统一集中在 `buildCallGraph` 中完成，避免重复逻辑。

在附录显示代码 4，为一个比较复杂的嵌套调用，且包括复杂类型匹配等，生成调用图如图 7 所示，可以看到程序生成正确，且能正确处理语法和语义分析。

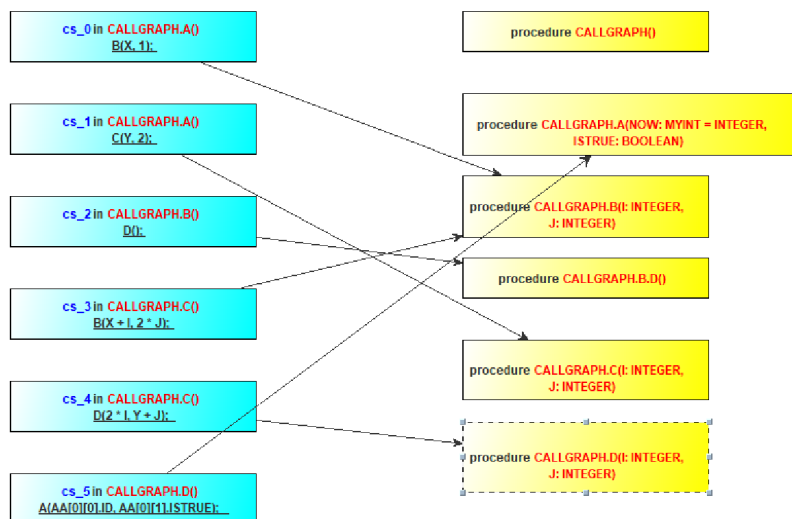


图 7: 错误恢复

8 讨论不同生成工具的差异

8.1 JavaCUP 和 GNU Bison

JavaCUP 和 GNU Bison 都是经典的 LALR(1) 语法分析器生成器。它们的核心功能相似：将基于上下文无关文法的描述文件转换为能够执行语法分析的源代码（JavaCUP 生成 Java 代码，Bison 生成 C/C++ 代码）。

语法文件结构对比 GNU Bison 通常采用经典的 Yacc 风格，分为声明、规则和辅助代码区。JavaCUP 虽然结构类似，但更强调 Java 的对象和类型安全。

特性	JavaCUP (.cup)	GNU Bison (.y)
类型处理	依赖 Java 泛型，显式类型声明	使用 %union 或 %define api.value.type
优先级定义	%precedence 机制	%left, %right, %nonassoc
错误恢复	error 方法	yyerrok 宏
动作嵌入	{: ... :} 块	{ ... } 块

表 4: JavaCUP 与 GNU Bison 的核心差异对比

类型安全与 Java 集成 JavaCUP 是为 Java 量身定做的，其语法规则中对非终结符的类型定义必须符合 Java 的静态类型系统。相比之下，Bison 原生为 C/C++ 设计，主要通过 Union 管理内存和类型。

```
===== Testing Parser =====  
  
[PASS] gcd008.obr  
[PASS] gcd009.obr  
[PASS] gcd010.obr  
[PASS] gcd011.obr  
[PASS] gcd012.obr  
[PASS] gcd013.obr  
[PASS] gcd014.obr  
[PASS] gcd015.obr  
[PASS] gcd016.obr  
[PASS] gcd017.obr  
[PASS] gcd018.obr  
[PASS] gcd019.obr  
[PASS] gcd020.obr  
[PASS] gcd021.obr  
  
Results: 14 passed, 0 failed out of 14 tests.  
  
===== Test Complete =====
```

图 8: 异常处理批量测试

语法嵌入逻辑 JavaCUP 中语法定义文件与 Java 类系统集成度高, 生成的代码更具可读性; 而 GNU 侧重于生成高性能的 C/C++ 状态机代码。

8.2 JavaCUP 和 JavaCC

JavaCUP 的语法分析方式为 LALR(1), 而 JavaCC 使用 LL(k) 方式。LL(k) 自顶向下编写更直观, 自底向上的 LALR(1) 能处理更复杂的文法。

9 自动测试

实现了一个异常测试类, 能够获取 testcases/ 文件下的 .obr 文件, 并根据第一行的期望异常和行数, 与 Parser 输出到 System.err 流的异常进行正则匹配, 自动测试, 测试结果如图 8 所示。

10 附录

```
(* An Oberon-0 demonstrating the usage of a call graph. *)  
MODULE CallGraph;  
  TYPE  
    myint=INTEGER;  
    myrecord=RECORD  
      id: myint;  
      istrue: BOOLEAN;  
    END;  
    myarray=ARRAY 10 OF myrecord;
```

```
VAR x, y: INTEGER;
PROCEDURE a(now: mYint; istrue: BOOLEAN);
BEGIN
    b(x, 1);
    c(y, 2);
END a;
PROCEDURE b(i: INTeGER; j: INTEGEr);
    PROCEDURE d();
    BEGIN
    END d;
BEGIN
    d();
    x := i + j;
END b;
PROCEDURE c(i: INTEGER; j: INTEGER);
BEGIN
    b(x + i, 2 * j);
    d(2 * i, y + j);
END c;
PROCEDURE d(i: INTEGER; j: INTEGER);
    VAR aa: ARRAY 10 OF myarray;
BEGIN
    a(aa[0][0].id, aa[0][1].istrue);
    y := i * j;
END d;
END CallGraph.
```

Listing 4: 调用图示例程序